SPI Report

Authors:

Elias Dahl – Abstract, Design, Discussion / Conclusion

Gabriel Minton – Introduction, Operation / Testing

Date: 11/13/2025

Contents

1 Abstract **2**

2 Introduction **2**

3 Design **2**

3.1 Timing 2

3.2 Finite State Machine 4

3.3 System Verilog Design 5

4 Operation / Testing **7**

5 Discussion / Conclusion **9**

6 Appendix **10**

1 Abstract

In this report, the authors discuss their design and implementation of a Serial Peripheral Interface (SPI), for use with a LTC1865L analog-to-digital Converter(ADC) and LTC1654 digital-to-analog converter (DAC). From the ADC and DAC datasheets, a timing diagram was made to inform the SPI module's design. Next, a finite state machine (FSM) for the SPI was developed and implemented in System Verilog (SV). The module was then tested with a simulated ADC and DAC in two SV testbenches. Finally, two identical instantiations of the module were implemented on on a Nexys A7 100-T FPGA, driven by a C program running on the board's Microblaze processor. The SPI instance connected to the DAC was tested with two input sine waves, with the output measured on an oscilloscope. Next, the SPI connected to the ADC was connected to a function generator and the output was printed to the C console. Additionally, both SPIs were tested by passing the ADC output though its SPI to the DAC's SPI, and measuring the DAC output on the oscilloscope.

2 Introduction

In this project, we are continuing our goal of audio signal processing by implementing an SPI interface. The SPI (Serial Peripheral Interface) does is not standardized like USB, so we need to specify all of the timing requirements and signals needed. For our purposes, we will need two instances of the SPI module in System Verilog (SV) for the ADC (analog-to-digital converter) and DAC (digital-to-analog converter). As the names suggest, these two pieces of hardware allow for input and output of an analog signal from a digital system. Since there is standard for SPI, the ADC (LTC1865L [1]) and DAC (LTC1654 [2]) have different requirements to run properly. For simplicity, we created a single SPI module that works with both chips.

The following will discuss the design process and considerations. Then, the operation and testing regiment used on SPI blocks. Finally, we discuss the design's quality and give final thoughts about the overall project. Related code used for testing purposes can be found in the appendix.

3 Design

The design process for the SPI had three phases: making the timing diagram, designing the FSM, and writing the System Verilog module.

3.1 Timing

From the project description, we knew that our SPI would need five external signals, the 30MHz main clock and the four ADC/DAC peripheral signals: CONV/CS, SCK, SDI, and SDO. CONV, SCK, and SDI are all input signals to the peripherals, so they would need to be outputs of the SPI. Conversely, SDO is a peripheral output, and would need to be a SPI input. To determine how the SDI outputs should be driven and when SDO should be read, we found the relevant worst case timings from the ADC [1] and DAC [2] datasheets and used them to draw a timing diagram for the SPI, shown below in Figure 1.

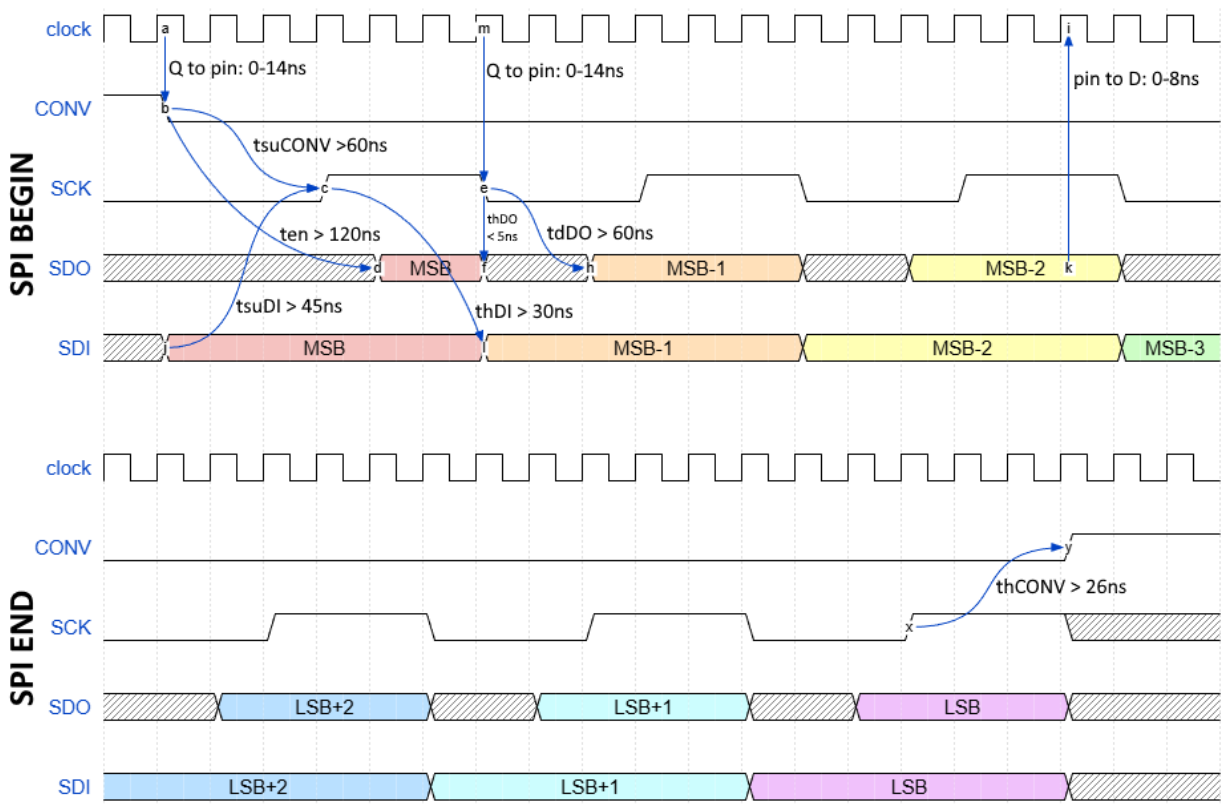


Figure 1: SPI timing diagram. The top set of signals show the timing right after CONV goes low, while the bottom set shows the timing right before CONV goes high.

Making the diagram in Figure 1 made many of the SPI module design decisions for us. The first was the SCK period. From the ADC [1] and DAC [2] datasheets, the ADC can at maximum have an SCK frequency of 8MHZ, while the DAC can use up to 25MHz. As our SPI needed to support both peripherals, 8MHz was our theoretical max. Additionally, the SCK high and low time for the ADC each need to be at least 45% of the SCK period at 8MHz, which is:

$$\frac{1}{8MHz} \times 0.45 = 56.35ns$$

Given the output Q to pin time of 0-14ns, an SCK high/low pulse changing every two main clock cycles could have times as short as $66.6ns - 14ns = 52.6ns < 56.35ns$, so the pulse width for SCK must be at least three clock cycles, giving our SCK a six clock cycle period.

The second design decision made clear from Figure 1 was when SDO should be read. While every bit besides the most significant bit(MSB) of SDO is valid for a few clock cycles, the MSB is is only valid for two. We decided to read SDO on the third/last main clock cycle of SCK being high, as the MSB is clearly valid at that point, as is every other bit.

The third design choice made from creating Figure 1 was when to update SDI. As seen in Figure 1, SDI needs to be valid at least 45ns before the rising edge of SCK and held valid for at least 30ns after. We found that setting the MSB of SDI when CONV falls and then updating SDI with the falling edge of SCK was an easy way to meet these requirements.

3.2 Finite State Machine

After gaining insights from our timing diagram, we began designing our FSM. After a few iterations, we settled on the version shown below in Figure 2, with signal names updated to match our System Verilog implementation.

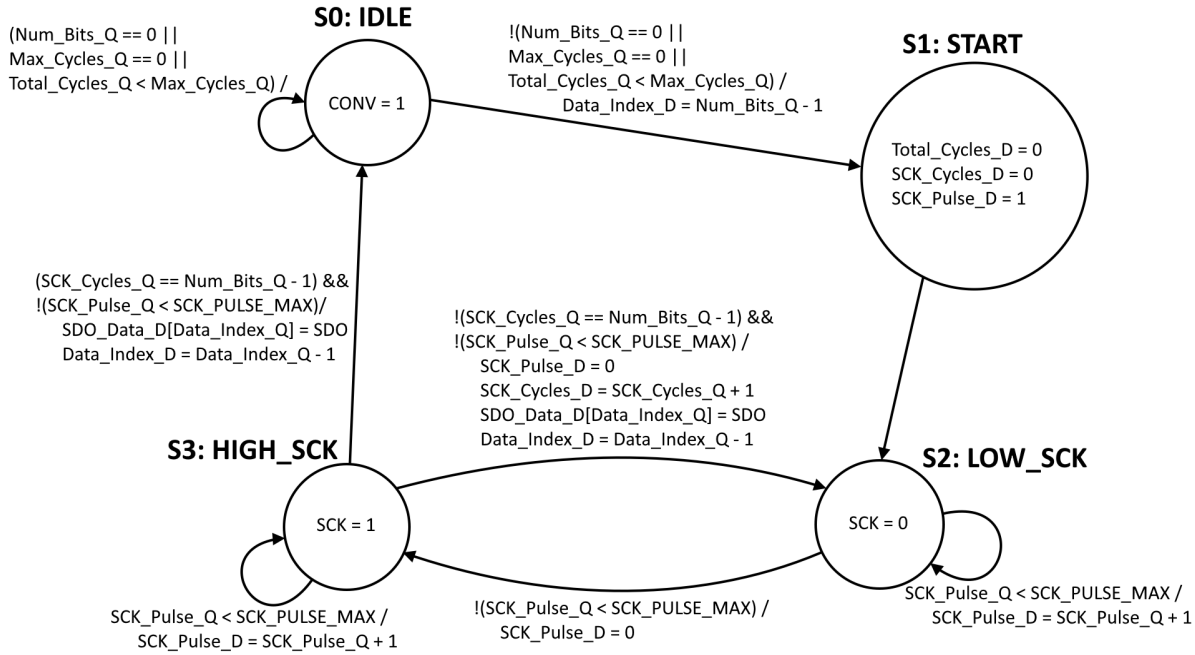


Figure 2: SPI FSM

The first state of the FSM is IDLE, which primarily holds CONV high. For the FSM to be able to leave IDLE, both the number of bits being processed and the maximum number of cycles, which determines the sample rate being used, need to be set to non-zero values. Setting both of these values allows the SPI to begin normal operation, and setting either back to 0 will "disable" the SPI. Then, if the total number of cycles since the last time the SPI exited IDLE(or since the SPI was turned on) equals Max_Cycles, the FSM moves from IDLE to START, and updates Data_Index to be Num_Bits - 1, which makes the MSB of SDI valid for the peripheral.

The next state after IDLE is START, which initializes the counters for the next loop of the FSM. Note that SCK_Pulse_D is set to 1 in START as it acts as the first of three main clock cycles for the first low section of SCK.

After START, the FSM begins moving between LOW_SCK and HIGH_SCK, which mainly hold SCK low and high respectively. The FSM remains in each of these states for three cycles before moving to the other. This pattern happens a number of times equal to the number of bits being processed by the SPI. Note that two important operations happen when HIGH_SCK is exited for IDLE or LOW_SCK: the current SDO bit is read, and Data_Index is decremented, which updates SDI and determines where the next SDO bit is saved.

3.3 System Verilog Design

Due to the relative simplicity of the SPI's design, we opted to jump right into writing the SV module, rather than writing a testbench first. Besides the combinational logic for the FSM, the other main aspect of the module was how we would read from and write to it, and what we would need to read and write. To perform the reads and writes, we used an AXI4Lite supporter in conjunction with a simple bus instantiated in the SPI module. The defined addresses and what data they are used for is shown below in 1.

Address	Type	Purpose
'h0	Write	Writes to the Num_Bits register to set the number of bits processed by the SPI
'h10	Write	Writes to the Max_Cycles register to set the sample rate of the SPI
'h4	Write	Writes to the SDI_Data register, where the next SDI to be processed is held
'h8	Read	Reads from the SDO_Data register, which holds the last SDO that was processed
'hc	Read	Reads CONV, for checking if the SPI is active

Table 1: Defined addresses and their purposes.

The full SV SPI module can be seen in Listing 1 of the appendix.

To test the SPI module, we wrote two SV testbenches, one for use with an ADC and one for use with a DAC, seen in Listings 2 and 3 of the appendix, discussed more in Operation / Testing below. The only design change that had to be made during testing was writing a line of code that we had forgotten to write, only having commented in the module that we should implement it.

After testing in simulation, we created a block diagram with two instantiations of the SPI module to implement in hardware, one for the ADC and one for the DAC, shown below in Figure 3.

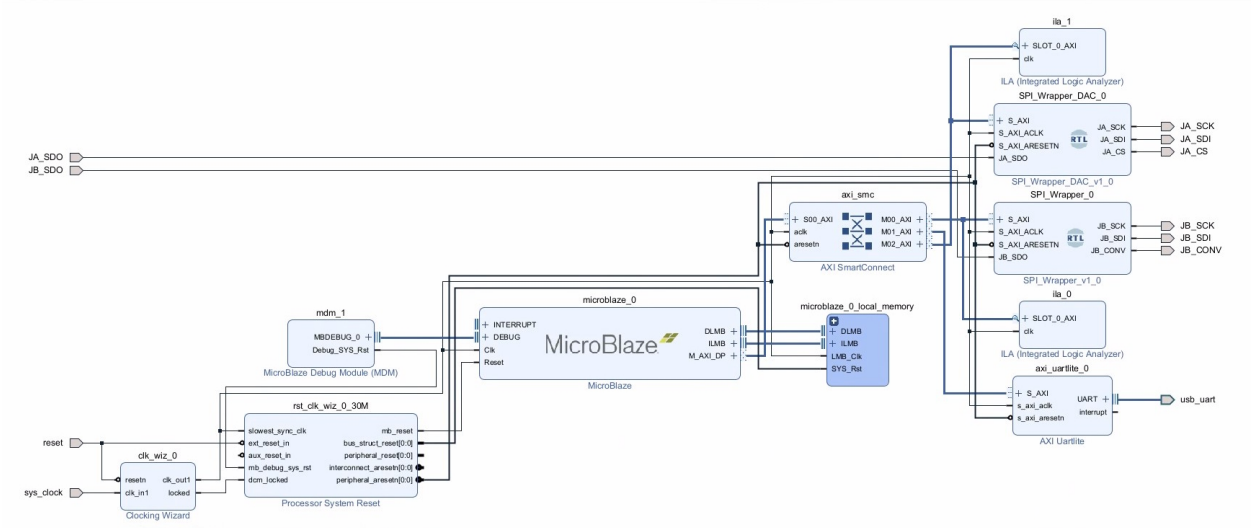


Figure 3: Vivado Block Diagram

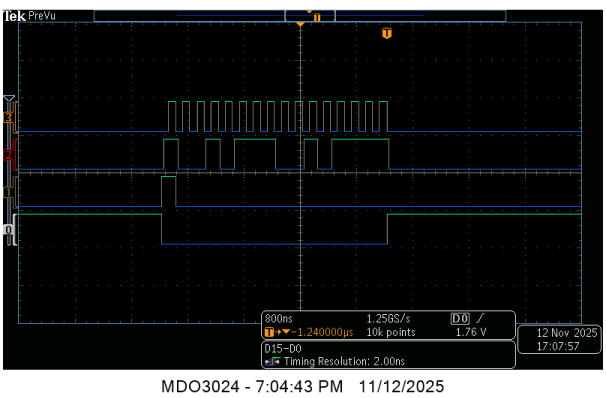
As seen in Figure 3, each SPI module in the block diagram had its own set of input/output signals, as well as their own integrated logic analyzers(ILA) for so their signals could be observed during testing.

For running and testing the hardware implementation of the SPI modules, we wrote a C program based on our SV testbenches, shown in full in Listing 4 of the appendix. The only errors we found during this stage were in our C code, such as using && instead of & when performing bit-masking operations. The hardware testing process is discussed in depth below in Operation / Testing.

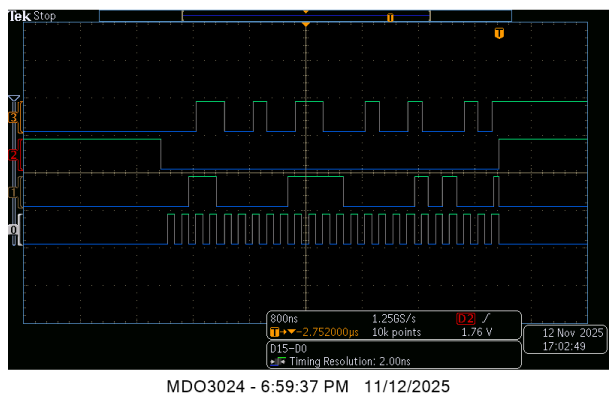
4 Operation / Testing

For this project, our main design considerations were having the SPI peripheral run at the specified frequency (25 KS / s / channel) and that input and output signals have equivalent amplitude and frequency. To accomplish this, we tested the SPI modules we designed in three phases, ADC, DAC, and ADC → DAC.

First, we tested the ADC using a sample created in Excel. This was done through an SV simulation testbench by saving the output samples and plotting the samples in Excel to ensure the frequency was properly maintained by the ADC conversion. After positive results from the testbench, we moved to testing on hardware. To control the hardware, we wrote a section of C code to control the FPGA board's MicroBlaze. Figure 4 shows the activity of the SPI pins of the ADC. Since D1:SDI and D2:SDO are the input signifying the next channel to be read and the converted sample, respectively. D3:SCK goes for 16 clock cycles, like the ADC expects. D0:CONV stays low for the specified amount of time.



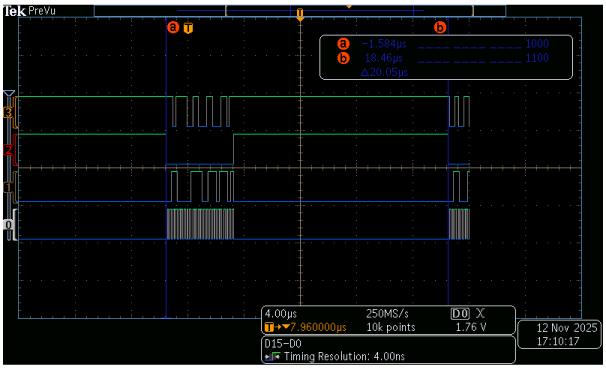
(a) ADC pins



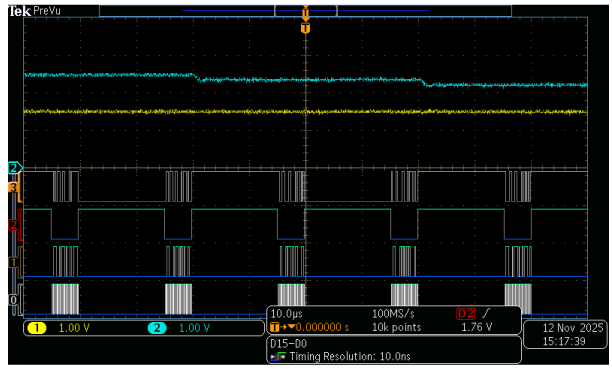
(b) DAC pins

Figure 4: SPI signals read on the oscilloscope for the ADC and DAC peripherals.

Second, the DAC was tested using the sample signals made in Excel in the SV testbench. After this was successful the testing continued on hardware, seeing that the output wave on oscilloscope has the same frequency and amplitude as the input wave. Figure 4 shows a sample SPI interaction for the DAC. D0:SCK, D1:SDI, D2:C̃S/LD, D3:SDO. The next portion of the testing is ensuring that the SPI is running at 50kHz. Figure 5 shows that the time of a full CS cycle on the DAC is $T = 20.05\mu s$, which is significantly close to our derired $T = 20\mu s$. This figure also shows several cycles of CS with the analog outputs. The blue curve (L-channel) changes every 2 SPI cycles, which is the desired functionality, since both audio channels are being processed concurrently.



(a) Full SPI interaction cycle $T = 20.05\mu s$.

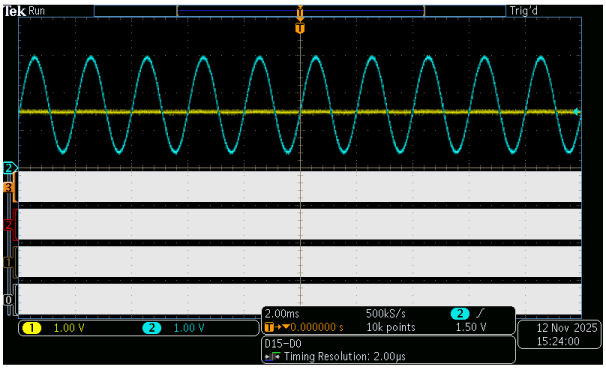


(b) Several SPI cycles with output waves

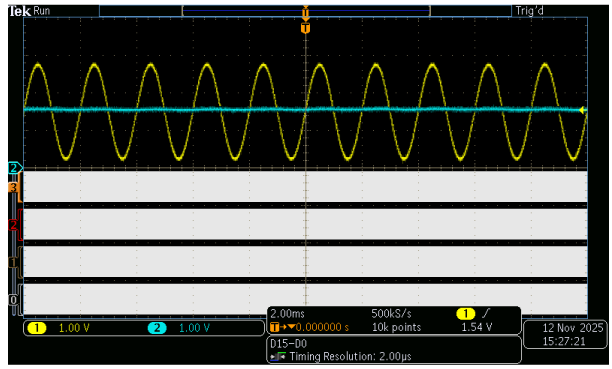
Figure 5: Oscilloscope readings for testing SPI DAC timings.

The final stage of testing for the SPI module is testing the DAC and ADC together. In ensuring a signal can be input through the ADC and output through the DAC, we are able to see that the SPI module functions as expected. To do this, we connect the ADC to the desktop function generator and DAC to the oscilloscope. A new C script is written up to combine the functionalities of the ADC and DAC, working together in harmony. Figure 6 gives the two SPI modules working together with a variety of input signals. As you can see, each plot has a different combination of input signals for each channel, ranging from constant DC signals to sine waves to the trigger output of the function generator. The plots in Figure 6 also show that each audio channel is processed independently, which is expected by our design restraints.

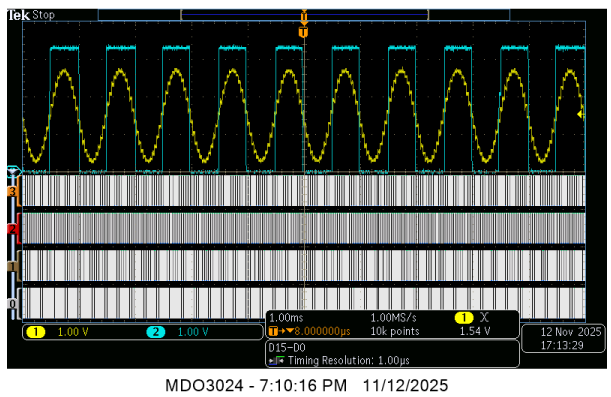
Overall, the operation of the SPI modules is well within the given range of design parameters. The ADC and DAC are able to process 2-channel audio concurrently, running at a specified sample rate. The ability to get analog signals into and out of a digital system is a major milestone for the final audio equalizer project.



(a) Right - DC, Left - 500 Hz



(b) Right - 500 Hz, Left - DC



(c) Right - 1 kHz, Left - FGen Trigger

Figure 6: Oscilloscope readings showing analog outputs (R (yellow) / L (blue)) and digital signals to the DAC.

5 Discussion / Conclusion

Unfortunately for our egos, we have no previous software implementation of the SPI to compare its performance to. We must simply be satisfied that it works. However, looking back upon the design process, some design choices we made could have been done differently that could have improved the quality of the design, depending on your perspective and what your design goals are.

The design choices we made that could have been done differently are all related to how the user interacts with the SPI. We chose to have the number of bits being used be a user configurable number up to 32 bits, the data width of the Axi4Lite Bus. This could allow the SPI to be used with other peripherals that use other numbers of bits other than 16 or 24. However, given the specific context of the project, we could have made the selection a simple toggle between the 16 and 24 bit modes. Similarly, we chose to have the SPI be enabled when both Num_Bits and Max_Cycles are set above 0, and disabled when either are set to 0. This kept the design simple, but it's less intuitive than if we had added a dedicated address and flip-flop for enabling the SPI. While we stand by our decisions in this case, there were other valid options available.

To conclude, our SPI module allowed us to easily read and write data to serial peripherals. This is

something that would have been a nightmare, if possible at all, to do though software. Through designing the SPI, we have yet again been reminded that working thoroughly though each step of the design process makes the next stage that much easier. Because we took our time getting our timing diagram and FSM right before moving onto writing the SV module, we ended up with only a single simulation bug. This carried over to our hardware implementation, where our only bugs were in our C code.

6 Appendix

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////
// Engineers: Elias Dahl, Gabriel Minton
// Create Date: 10/28/2025 01:09:32 PM
// Module Name: SPI
////////////////////////////////////////////////////////////////////////////////


module SPI_updated #
     (parameter C_S_AXI_ADDR_WIDTH = 6, C_S_AXI_DATA_WIDTH = 32, SCK_PULSE_MAX =
         2)
    (
        // Axi4Lite Bus
        input  logic  S_AXI_ACLK,
        input  logic  S_AXI_ARESETN,
        input  logic  [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR,
        input  logic  S_AXI_AWVALID,
        output logic  S_AXI_AWREADY,
        input  logic  [C_S_AXI_DATA_WIDTH-1:0] S_AXI_WDATA,
        input  logic  [3:0] S_AXI_WSTRB,
        input  logic  S_AXI_WVALID,
        output logic  S_AXI_WREADY,
        input  logic  [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_ARADDR,
        input  logic  S_AXI_ARVALID,
        output logic  S_AXI_ARREADY,
        output logic  [C_S_AXI_DATA_WIDTH-1:0] S_AXI_RDATA,
        output logic  [1:0] S_AXI_RRESP,
        output logic  S_AXI_RVALID,
        input  logic  S_AXI_RREADY,
        output logic  [1:0] S_AXI_BRESP,
        output logic  S_AXI_BVALID,
        input  logic  S_AXI_BREADY,
        // SPI pins
        output logic  SCK,
        output logic  SDI,
        output logic  CONV,
        input  logic  SDO
    );

     // Simple Bus Signals
        logic    [C_S_AXI_ADDR_WIDTH-1:0] wrAddr;
        logic    [C_S_AXI_DATA_WIDTH-1:0] wrData;
        logic    wr;
        logic    [C_S_AXI_ADDR_WIDTH-1:0] rdAddr;
        logic    [C_S_AXI_DATA_WIDTH-1:0] rdData;
        logic    rd;
```

```
    // Instantiate Supporter
     Axi4LiteSupporter #(.C_S_AXI_ADDR_WIDTH(C_S_AXI_ADDR_WIDTH),.
         C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH)) Axi4LiteSupporter1 (
        // Simple Bus
        .wrAddr(wrAddr),                    // output   [C_S_AXI_ADDR_WIDTH-1:0]
        .wrData(wrData),                    // output   [C_S_AXI_DATA_WIDTH-1:0]
        .wr(wr),                            // output
        .rdAddr(rdAddr),                    // output   [C_S_AXI_ADDR_WIDTH-1:0]
        .rdData(rdData),                    // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .rd(rd),                            // output
        // Axi4Lite Bus
        .S_AXI_ACLK(S_AXI_ACLK),            // input
        .S_AXI_ARESETN(S_AXI_ARESETN),      // input
        .S_AXI_AWADDR(S_AXI_AWADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_AWVALID(S_AXI_AWVALID),      // input
        .S_AXI_AWREADY(S_AXI_AWREADY),      // output
        .S_AXI_WDATA(S_AXI_WDATA),          // input    [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_WSTRB(S_AXI_WSTRB),          // input    [3:0]
        .S_AXI_WVALID(S_AXI_WVALID),        // input
        .S_AXI_WREADY(S_AXI_WREADY),        // output
        .S_AXI_ARADDR(S_AXI_ARADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_ARVALID(S_AXI_ARVALID),      // input
        .S_AXI_ARREADY(S_AXI_ARREADY),      // output
        .S_AXI_RDATA(S_AXI_RDATA),          // output   [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_RRESP(S_AXI_RRESP),          // output   [1:0]
        .S_AXI_RVALID(S_AXI_RVALID),        // output
        .S_AXI_RREADY(S_AXI_RREADY),        // input
        .S_AXI_BRESP(S_AXI_BRESP),          // output   [1:0]
        .S_AXI_BVALID(S_AXI_BVALID),        // output
        .S_AXI_BREADY(S_AXI_BREADY)         // input
        ) ;


//Internal signals
    logic [1:0] SCK_Pulse_D, SCK_Pulse_Q; // For changing SCK High to Low.
    logic [9:0] Total_Cycles_D, Total_Cycles_Q; // Counts up to Max_Cycles
    logic [4:0] SCK_Cycles_D, SCK_Cycles_Q; // Counts number of SCK cycles, up to
        the number of bits processed.
    logic [4:0] Num_Bits_D, Num_Bits_Q, //Number of bits to be processed: 16 or
        24, written by the manager.
              Data_Index_D, Data_Index_Q; //Index into SDI and SDO data registers.

    // Holds the number of clock cycles per sample. Needs to go up to 30,000 for
        lowest intended sample rate of 1kHz.
    logic [14:0] Max_Cycles_D, Max_Cycles_Q;

    logic [C_S_AXI_DATA_WIDTH-1:0] SDI_Data_D, SDI_Data_Q, //Data received from
        the manager to output on SDI
                                   SDO_Data_D, SDO_Data_Q; //Data converted from
                                       SDO input.
```

```
//FSM State
typedef enum logic [1:0] {IDLE, START, LOW_SCK, HIGH_SCK} statetype;
statetype nextState, currState;

// Sequential Logic
always_ff @ (posedge S_AXI_ACLK) begin
    if ( ! S_AXI_ARESETN) begin
        currState <= IDLE;
        SCK_Pulse_Q <= 0;
        Total_Cycles_Q <= 0;
        SCK_Cycles_Q <= 0;
        Num_Bits_Q <= 0;
        Max_Cycles_Q <= 0;
        SDI_Data_Q <= 0;
        SDO_Data_Q <= 0;
        Data_Index_Q <= 0;
    end else begin
        currState <= nextState;
        SCK_Pulse_Q <= SCK_Pulse_D;
        Total_Cycles_Q <= Total_Cycles_D;
        SCK_Cycles_Q <= SCK_Cycles_D;
        Num_Bits_Q <= Num_Bits_D;
        Max_Cycles_Q <= Max_Cycles_D;
        SDI_Data_Q <= SDI_Data_D;
        SDO_Data_Q <= SDO_Data_D;
        Data_Index_Q <= Data_Index_D;
    end
end

// Combinational Logic
always_comb begin
// Default Values.
    nextState = currState;
    SCK_Pulse_D = SCK_Pulse_Q;
    SCK_Cycles_D = SCK_Cycles_Q;
    Num_Bits_D = Num_Bits_Q;
    Max_Cycles_D = Max_Cycles_Q;
    SDI_Data_D = SDI_Data_Q;
    SDO_Data_D = SDO_Data_Q;
    Data_Index_D = Data_Index_Q;

    // Needs to be incremented every clock cycle.
    Total_Cycles_D = Total_Cycles_Q + 1;

    // SDI bit is changed by changing Data_Index in the FSM.
    SDI = SDI_Data_Q[Data_Index_Q];

    CONV = 0;
    SCK = 0;
    rdData = 0;


    // FSM
    case(currState)
        IDLE: begin
```

```
        CONV = 1;

        // Stay in Idle if the SPI hasn't been initialized or the
            peripheral is still converting.
        if (Num_Bits_Q == 0 || Max_Cycles_Q == 0 || Total_Cycles_Q <
            Max_Cycles_Q) begin
            nextState = IDLE;
        end

        // Exit IDLE otherwise
        else begin
            nextState = START;
            // Update Data_Index to its inital value so the First SDI bit
                is valid.
            Data_Index_D = Num_Bits_Q-1;
        end
    end

    // First third of the first low level of SCK.
    // Resets the Total_Cycles and SCK_Cycles counters.
    START: begin
        CONV = 0;
        Total_Cycles_D = 0;
        SCK_Cycles_D = 0;
        SCK_Pulse_D = 1;
        nextState = LOW_SCK;
    end

    LOW_SCK: begin
        SCK = 0;
        // SCK stays LOW for three clock cycles.
        if (SCK_Pulse_Q < SCK_PULSE_MAX) begin
            nextState = LOW_SCK;
            SCK_Pulse_D = SCK_Pulse_Q + 1;
        end
        else begin
            nextState = HIGH_SCK;
            SCK_Pulse_D = 0;
        end
    end

    HIGH_SCK: begin
        SCK = 1;
        //SCK stays high for three clock cycles.
        if (SCK_Pulse_Q < SCK_PULSE_MAX) begin
            nextState = HIGH_SCK;
            SCK_Pulse_D = SCK_Pulse_Q + 1;
        end

        // If in the last SCK cycle,
        // go back to IDLE after reading SDO.
        else if(SCK_Cycles_Q == Num_Bits_Q - 1) begin
            nextState = IDLE;
            // Read SDO
            SDO_Data_D[Data_Index_Q] = SDO;
```

```
                    Data_Index_D = Data_Index_Q - 1;
                end

                // Otherwise read SDO and go back to LOW_SCK
                else begin
                    nextState = LOW_SCK;
                    SCK_Pulse_D = 0;
                    SCK_Cycles_D = SCK_Cycles_Q + 1;
                    // Read SDO
                    SDO_Data_D[Data_Index_Q] = SDO;
                    Data_Index_D = Data_Index_Q - 1;
                end
            end

            default: begin
                nextState = IDLE;
            end
        endcase

// Read & Write Logic
       if(wr) begin
           case(wrAddr)
           // Write to Num_Bits
               'h0: begin
                    Num_Bits_D = wrData[4:0];
                end

            // Write to Max_Cycles
               'h10: begin
                    Max_Cycles_D = wrData[14:0];
                end

            // Write to SDI Data
               'h4: begin
                    SDI_Data_D = wrData;
                end

           // Any other wrAddr is invalid, so do nothing.
               default: begin
               end
           endcase
        end

        if (rd) begin
            case(rdAddr)
            // Read from SDO_Data
                'h8: begin
                    rdData = SDO_Data_Q;
                end

            // Read CONV for checking if the SPI is Active
                'hc: begin
                    rdData = {{C_S_AXI_DATA_WIDTH-1{1'b0}},CONV};
                end
```

```
            // Default case, do nothing.
                default: begin
                end
            endcase
        end
    end

endmodule
```

Listing 1: SPI SV module.

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////
// Engineers: Elias Dahl, Gabriel Minton
// Create Date: 10/30/2025 03:34:05 PM
// Module Name: SPI_ADC_TB
////////////////////////////////////////////////////////////////////////////////


module SPI_ADC_TB_updated();
parameter C_S_AXI_DATA_WIDTH = 32, C_S_AXI_ADDR_WIDTH = 6, CLK_PERIOD = 33.33;
// Axi4Lite signals
logic   S_AXI_ACLK ;
logic   S_AXI_ARESETN ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR ;
logic   S_AXI_AWVALID ;
logic   S_AXI_AWREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] S_AXI_WDATA ;
logic   [3:0] S_AXI_WSTRB ;
logic   S_AXI_WVALID ;
logic   S_AXI_WREADY ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_ARADDR ;
logic   S_AXI_ARVALID ;
logic   S_AXI_ARREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] S_AXI_RDATA ;
logic   [1:0] S_AXI_RRESP ;
logic   S_AXI_RVALID ;
logic   S_AXI_RREADY ;
logic   [1:0] S_AXI_BRESP ;
logic   S_AXI_BVALID ;
logic   S_AXI_BREADY ;

// Manager simple Bus signals
logic   [C_S_AXI_ADDR_WIDTH-1:0]    wrAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    wrData ;
logic                               wr ;
logic                               wrDone ;
logic   [C_S_AXI_ADDR_WIDTH-1:0]    rdAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    rdData ;
logic                               rd ;
logic                               rdDone ;

//SPI Signals
logic  SCK;
logic  SDI;
logic  CONV;
logic  SDO;

SPI_updated #(.C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH),.C_S_AXI_ADDR_WIDTH(
    C_S_AXI_ADDR_WIDTH)) SPI_1 (
         // Axi4LiteRegs Bus
        .S_AXI_ACLK(S_AXI_ACLK),            // input
        .S_AXI_ARESETN(S_AXI_ARESETN),      // input
        .S_AXI_AWADDR(S_AXI_AWADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_AWVALID(S_AXI_AWVALID),      // input
        .S_AXI_AWREADY(S_AXI_AWREADY),      // output
```

```
        .S_AXI_WDATA(S_AXI_WDATA),          // input    [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_WSTRB(S_AXI_WSTRB),          // input    [3:0]
        .S_AXI_WVALID(S_AXI_WVALID),        // input
        .S_AXI_WREADY(S_AXI_WREADY),        // output
        .S_AXI_ARADDR(S_AXI_ARADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_ARVALID(S_AXI_ARVALID),      // input
        .S_AXI_ARREADY(S_AXI_ARREADY),      // output
        .S_AXI_RDATA(S_AXI_RDATA),          // output   [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_RRESP(S_AXI_RRESP),          // output   [1:0]
        .S_AXI_RVALID(S_AXI_RVALID),        // output
        .S_AXI_RREADY(S_AXI_RREADY),        // input
        .S_AXI_BRESP(S_AXI_BRESP),          // output   [1:0]
        .S_AXI_BVALID(S_AXI_BVALID),        // output
        .S_AXI_BREADY(S_AXI_BREADY),         // input
        .SCK(SCK),
        .SDI(SDI),
        .CONV(CONV),
        .SDO(SDO)
);

Axi4LiteManager #(.C_M_AXI_ADDR_WIDTH(C_S_AXI_ADDR_WIDTH), .C_M_AXI_DATA_WIDTH(
    C_S_AXI_DATA_WIDTH)) Axi4LiteManager1
        (
        // Simple Bus
        .wrAddr(wrAddr),                    // input    [C_M_AXI_ADDR_WIDTH-1:0]
        .wrData(wrData),                    // input    [C_M_AXI_DATA_WIDTH-1:0]
        .wr(wr),                            // input
        .wrDone(wrDone),                    // output
        .rdAddr(rdAddr),                    // input    [C_M_AXI_ADDR_WIDTH-1:0]
        .rdData(rdData),                    // output   [C_M_AXI_DATA_WIDTH-1:0]
        .rd(rd),                            // input
        .rdDone(rdDone),                    // output
        // Axi4Lite Bus
        .M_AXI_ACLK(S_AXI_ACLK),            // input
        .M_AXI_ARESETN(S_AXI_ARESETN),      // input
        .M_AXI_AWADDR(S_AXI_AWADDR),        // output   [C_M_AXI_ADDR_WIDTH-1:0]
        .M_AXI_AWVALID(S_AXI_AWVALID),      // output
        .M_AXI_AWREADY(S_AXI_AWREADY),      // input
        .M_AXI_WDATA(S_AXI_WDATA),          // output   [C_M_AXI_DATA_WIDTH-1:0]
        .M_AXI_WSTRB(S_AXI_WSTRB),          // output   [3:0]
        .M_AXI_WVALID(S_AXI_WVALID),        // output
        .M_AXI_WREADY(S_AXI_WREADY),        // input
        .M_AXI_ARADDR(S_AXI_ARADDR),        // output   [C_M_AXI_ADDR_WIDTH-1:0]
        .M_AXI_ARVALID(S_AXI_ARVALID),      // output
        .M_AXI_ARREADY(S_AXI_ARREADY),      // input
        .M_AXI_RDATA(S_AXI_RDATA),          // input    [C_M_AXI_DATA_WIDTH-1:0]
        .M_AXI_RRESP(S_AXI_RRESP),          // input    [1:0]
        .M_AXI_RVALID(S_AXI_RVALID),        // input
        .M_AXI_RREADY(S_AXI_RREADY),        // output
        .M_AXI_BRESP(S_AXI_BRESP),          // input    [1:0]
        .M_AXI_BVALID(S_AXI_BVALID),        // input
        .M_AXI_BREADY(S_AXI_BREADY)         // output
        );

    AdcTester AdcTester1 (
```

```
.SCK(SCK),
.SDI(SDI),
.CS_(CONV),
.SDO(SDO)
);

parameter CLK_PERIOD_2 = (CLK_PERIOD)/2;
// Generate Clock
always begin
    #(CLK_PERIOD_2) S_AXI_ACLK = ~S_AXI_ACLK;
end

// create memory arrays
logic [C_S_AXI_DATA_WIDTH-1:0] outputL [0:999];
logic [C_S_AXI_DATA_WIDTH-1:0] outputR [0:999];
int i;
logic r_CONV;

initial begin
    // Initialize
    S_AXI_ARESETN = 0;
    S_AXI_ACLK = 0;
    wr = 0;
    wrAddr = 0;
    wrData = 0;
    rd = 0;
    rdAddr = 0;
    r_CONV = 0;

    //Generate Reset
    #(CLK_PERIOD_2 + 2) S_AXI_ARESETN = 1;
    #(CLK_PERIOD*10);

    // Addr of Num_Bits
    wrAddr = 'h0;
    wrData = 'd16;
    wr = 1;
    #(CLK_PERIOD*2);
    wr = 0;
    while(!wrDone) begin
        #(CLK_PERIOD);
    end

    // Addr of Max_Cycles
    wrAddr = 'h10;
    wrData = 'd600; // 25kHz sample rate.
    wr = 1;
    #(CLK_PERIOD*2);
    wr = 0;
    while(!wrDone) begin
        #(CLK_PERIOD);
    end


    rdAddr = 'hc;
```

```
        rd = 1;
        #(CLK_PERIOD*2);
        rd = 0;
        while(!rdDone)begin
            #(CLK_PERIOD);
        end

        //Wait for SPI cycle to discard first value.
        r_CONV = rdData[0];

        while(r_CONV == 1)begin
            rd = 1;
            #(CLK_PERIOD*2);
            rd = 0;
            while(!rdDone)begin
                #(CLK_PERIOD);
            end
            r_CONV = rdData[0];
        end

        while(r_CONV == 0)begin
            rd = 1;
            #(CLK_PERIOD*2);
            rd = 0;
            while(!rdDone)begin
                #(CLK_PERIOD);
            end
            r_CONV = rdData[0];
        end

        for(i = 0; i < 2000; i++) begin
            // Write SDI
            wrAddr ='h4;
            if(i%2 == 0) begin
                wrData = 'h8000; // single ended, channel 0;

            end
            else begin
                wrData = 'hc000; // single ended, channel 1;
            end

            wr = 1;
            #(CLK_PERIOD*2);
            wr = 0;

            while(!wrDone) begin
                #(CLK_PERIOD);
            end

            //Wait for SPI to be inactive before reading next sample.
            rdAddr = 'hc;
            while(r_CONV == 1)begin
                rd = 1;
                #(CLK_PERIOD*2);
                rd = 0;
```

```
                while(!rdDone)begin
                    #(CLK_PERIOD);
                end
                r_CONV = rdData[0];
            end

            while(r_CONV == 0)begin
                rd = 1;
                #(CLK_PERIOD*2);
                rd = 0;
                while(!rdDone)begin
                    #(CLK_PERIOD);
                end
                r_CONV = rdData[0];
            end

            // Read SDO
            rdAddr = 'h8;
            rd = 1;
            #(CLK_PERIOD*2);
            rd = 0;
            while(!rdDone) begin
                #(CLK_PERIOD);
            end

            if(i%2 == 1) begin
                outputL[i/2] = rdData; // single ended, channel 0;
            end
            else begin
                outputR[i/2] = rdData; // single ended, channel 1;
            end
        end

        $display("Output L: \n");
        for(i=0; i<1000; i++) begin
            $display("%h \n",(outputL[i]));
        end
        $display("Output R: \n");
        for(i=0; i<1000; i++) begin
            $display("%h \n",(outputR[i]));
        end
        $stop;
    end
endmodule
```

Listing 2: SPI ADC Testbench.

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////
// Engineers: Elias Dahl, Gabriel Minton
// Create Date: 10/30/2025 03:34:05 PM
// Module Name: SPI_ADC_TB
////////////////////////////////////////////////////////////////////////////////


module SPI_DAC_TB_updated();
parameter C_S_AXI_DATA_WIDTH = 32, C_S_AXI_ADDR_WIDTH = 6, CLK_PERIOD = 33.33;
// Axi4Lite signals
logic   S_AXI_ACLK ;
logic   S_AXI_ARESETN ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_AWADDR ;
logic   S_AXI_AWVALID ;
logic   S_AXI_AWREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] S_AXI_WDATA ;
logic   [3:0] S_AXI_WSTRB ;
logic   S_AXI_WVALID ;
logic   S_AXI_WREADY ;
logic   [C_S_AXI_ADDR_WIDTH-1:0] S_AXI_ARADDR ;
logic   S_AXI_ARVALID ;
logic   S_AXI_ARREADY ;
logic   [C_S_AXI_DATA_WIDTH-1:0] S_AXI_RDATA ;
logic   [1:0] S_AXI_RRESP ;
logic   S_AXI_RVALID ;
logic   S_AXI_RREADY ;
logic   [1:0] S_AXI_BRESP ;
logic   S_AXI_BVALID ;
logic   S_AXI_BREADY ;

// Manager simple Bus signals
logic   [C_S_AXI_ADDR_WIDTH-1:0]    wrAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    wrData ;
logic                               wr ;
logic                               wrDone ;
logic   [C_S_AXI_ADDR_WIDTH-1:0]    rdAddr ;
logic   [C_S_AXI_DATA_WIDTH-1:0]    rdData ;
logic                               rd ;
logic                               rdDone ;

//SPI Signals
logic  SCK;
logic  SDI;
logic  CONV;
logic  SDO;

SPI_updated #(.C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH),.C_S_AXI_ADDR_WIDTH(
    C_S_AXI_ADDR_WIDTH)) SPI_1 (
         // Axi4LiteRegs Bus
        .S_AXI_ACLK(S_AXI_ACLK),            // input
        .S_AXI_ARESETN(S_AXI_ARESETN),      // input
        .S_AXI_AWADDR(S_AXI_AWADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_AWVALID(S_AXI_AWVALID),      // input
        .S_AXI_AWREADY(S_AXI_AWREADY),      // output
```

```
        .S_AXI_WDATA(S_AXI_WDATA),          // input    [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_WSTRB(S_AXI_WSTRB),          // input    [3:0]
        .S_AXI_WVALID(S_AXI_WVALID),        // input
        .S_AXI_WREADY(S_AXI_WREADY),        // output
        .S_AXI_ARADDR(S_AXI_ARADDR),        // input    [C_S_AXI_ADDR_WIDTH-1:0]
        .S_AXI_ARVALID(S_AXI_ARVALID),      // input
        .S_AXI_ARREADY(S_AXI_ARREADY),      // output
        .S_AXI_RDATA(S_AXI_RDATA),          // output   [C_S_AXI_DATA_WIDTH-1:0]
        .S_AXI_RRESP(S_AXI_RRESP),          // output   [1:0]
        .S_AXI_RVALID(S_AXI_RVALID),        // output
        .S_AXI_RREADY(S_AXI_RREADY),        // input
        .S_AXI_BRESP(S_AXI_BRESP),          // output   [1:0]
        .S_AXI_BVALID(S_AXI_BVALID),        // output
        .S_AXI_BREADY(S_AXI_BREADY),         // input
        .SCK(SCK),
        .SDI(SDI),
        .CONV(CONV),
        .SDO(SDO)
);

Axi4LiteManager #(.C_M_AXI_ADDR_WIDTH(C_S_AXI_ADDR_WIDTH), .C_M_AXI_DATA_WIDTH(
    C_S_AXI_DATA_WIDTH)) Axi4LiteManager1
        (
        // Simple Bus
        .wrAddr(wrAddr),                    // input    [C_M_AXI_ADDR_WIDTH-1:0]
        .wrData(wrData),                    // input    [C_M_AXI_DATA_WIDTH-1:0]
        .wr(wr),                            // input
        .wrDone(wrDone),                    // output
        .rdAddr(rdAddr),                    // input    [C_M_AXI_ADDR_WIDTH-1:0]
        .rdData(rdData),                    // output   [C_M_AXI_DATA_WIDTH-1:0]
        .rd(rd),                            // input
        .rdDone(rdDone),                    // output
        // Axi4Lite Bus
        .M_AXI_ACLK(S_AXI_ACLK),            // input
        .M_AXI_ARESETN(S_AXI_ARESETN),      // input
        .M_AXI_AWADDR(S_AXI_AWADDR),        // output   [C_M_AXI_ADDR_WIDTH-1:0]
        .M_AXI_AWVALID(S_AXI_AWVALID),      // output
        .M_AXI_AWREADY(S_AXI_AWREADY),      // input
        .M_AXI_WDATA(S_AXI_WDATA),          // output   [C_M_AXI_DATA_WIDTH-1:0]
        .M_AXI_WSTRB(S_AXI_WSTRB),          // output   [3:0]
        .M_AXI_WVALID(S_AXI_WVALID),        // output
        .M_AXI_WREADY(S_AXI_WREADY),        // input
        .M_AXI_ARADDR(S_AXI_ARADDR),        // output   [C_M_AXI_ADDR_WIDTH-1:0]
        .M_AXI_ARVALID(S_AXI_ARVALID),      // output
        .M_AXI_ARREADY(S_AXI_ARREADY),      // input
        .M_AXI_RDATA(S_AXI_RDATA),          // input    [C_M_AXI_DATA_WIDTH-1:0]
        .M_AXI_RRESP(S_AXI_RRESP),          // input    [1:0]
        .M_AXI_RVALID(S_AXI_RVALID),        // input
        .M_AXI_RREADY(S_AXI_RREADY),        // output
        .M_AXI_BRESP(S_AXI_BRESP),          // input    [1:0]
        .M_AXI_BVALID(S_AXI_BVALID),        // input
        .M_AXI_BREADY(S_AXI_BREADY)         // output
        );

    DacTester DacTester1 (
```

```
.SCK(SCK),
.SDI(SDI),
.CS_(CONV),
.SDO(SDO)
);

parameter CLK_PERIOD_2 = (CLK_PERIOD)/2;
// Generate Clock
always begin
    #(CLK_PERIOD_2) S_AXI_ACLK = ~S_AXI_ACLK;
end

// create memory arrays

logic [15:0] inputL [0:999];
logic [15:0] inputR [0:999];

int i;
logic r_CONV;
logic [3:0] control;
logic [3:0] address;
logic [23:0] SDI_DATA;

initial begin
    $readmemh("SineWaveL.mem",inputL);
    $readmemh("SineWaveR.mem",inputR);
    // Initialize
    S_AXI_ARESETN = 0;
    S_AXI_ACLK = 0;
    wr = 0;
    wrAddr = 0;
    wrData = 0;
    rd = 0;
    rdAddr = 0;
    r_CONV = 0;


    //Generate Reset
    #(CLK_PERIOD_2 + 2) S_AXI_ARESETN = 1;
    #(CLK_PERIOD*10);

    // Addr of Num_Bits
    wrAddr = 'h0;
    wrData = 'd24;
    wr = 1;
    #(CLK_PERIOD*2);
    wr = 0;
    while(!wrDone) begin
        #(CLK_PERIOD);
    end

    // Addr of Max_Cycles
    wrAddr = 'h10;
    wrData = 'd600; // 25kHz sample rate.
    wr = 1;
```

```
#(CLK_PERIOD*2);
wr = 0;
while(!wrDone) begin
    #(CLK_PERIOD);
end

control = 'b0011; //3


for(i=0; i<2000; i++) begin
    if (i%2 == 0) begin
        address = 0;
        SDI_DATA = {control, address, inputL[i/2]};
    end
    else begin
        address = 1;
        SDI_DATA = {control, address, inputR[i/2]};
    end

    // Write SDI
    wrAddr = 'h4;
    wrData = SDI_DATA;
     wr = 1;
    #(CLK_PERIOD*2);
    wr = 0;
    while(!wrDone) begin
        #(CLK_PERIOD);
    end

    //Wait for SPI to be inactive before writing next sample.
    r_CONV = rdData[0];
    while(r_CONV == 1)begin
        rd = 1;
        #(CLK_PERIOD*2);
        rd = 0;
        while(!rdDone)begin
            #(CLK_PERIOD);
        end
        r_CONV = rdData[0];
    end

    while(r_CONV == 0)begin
        rd = 1;
        #(CLK_PERIOD*2);
        rd = 0;
        while(!rdDone)begin
            #(CLK_PERIOD);
        end
        r_CONV = rdData[0];
    end

end
```

```
        $stop;
    end
endmodule
```

Listing 3: SPI DAC Testbench.

```c
/*****************************************************************************
 * Copyright (C) 2023 Advanced Micro Devices, Inc. All Rights Reserved.
 * SPDX-License-Identifier: MIT
 ******************************************************************************/
/*
 * main.c
 *
 * This application configures UART 16550 to baud rate 9600.
 * PS7 UART (Zynq) is not initialized by this application, since
 * bootrom/bsp configures it to baud rate 115200
 *
 * ----------------------------------------------
 * | UART TYPE   BAUD RATE                        |
 * ----------------------------------------------
 *   uartns550   9600
 *   uartlite    Configurable only in HW design
 *   ps7_uart    115200 (configured by bootrom/bsp)
 */

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "sin_input_1.h"
#include "sin_input_2.h"

#define ADC_BASE_ADDR 0x10000
#define DAC_BASE_ADDR 0x4000

#define ADC_NUM_BITS_ADDR (ADC_BASE_ADDR)
#define ADC_MAX_CYCLES_ADDR (ADC_BASE_ADDR + 0x10)
#define ADC_SDI_DATA_ADDR (ADC_BASE_ADDR + 0x4)
#define ADC_SDO_DATA_ADDR (ADC_BASE_ADDR + 0x8)
#define ADC_CONV_ADDR (ADC_BASE_ADDR + 0xC)

#define DAC_NUM_BITS_ADDR (DAC_BASE_ADDR)
#define DAC_MAX_CYCLES_ADDR (DAC_BASE_ADDR + 0x10)
#define DAC_SDI_DATA_ADDR (DAC_BASE_ADDR + 0x4)
#define DAC_SDO_DATA_ADDR (DAC_BASE_ADDR + 0x8)
#define DAC_CONV_ADDR (DAC_BASE_ADDR + 0xC)

volatile int *ADC_NUM_BITS_PTR = (int *) ADC_NUM_BITS_ADDR;
volatile int *ADC_MAX_CYCLES_PTR = (int *) ADC_MAX_CYCLES_ADDR;
volatile int *ADC_SDI_DATA_PTR = (int *) ADC_SDI_DATA_ADDR;
volatile int *ADC_SDO_DATA_PTR = (int *) ADC_SDO_DATA_ADDR;
volatile int *ADC_CONV_PTR = (int *) ADC_CONV_ADDR;

volatile int *DAC_NUM_BITS_PTR = (int *) DAC_NUM_BITS_ADDR;
volatile int *DAC_MAX_CYCLES_PTR = (int *) DAC_MAX_CYCLES_ADDR;
volatile int *DAC_SDI_DATA_PTR = (int *) DAC_SDI_DATA_ADDR;
volatile int *DAC_SDO_DATA_PTR = (int *) DAC_SDO_DATA_ADDR;
volatile int *DAC_CONV_PTR = (int *) DAC_CONV_ADDR;

// Size of input/output arrays
#define NUM_SAMPLES 1000

//600 cycles for 25kHz sample rate
#define MAX_CYCLES 600

int rConv;

// Use with Run_ADC.
//Has to be commented out due to limted memory.
//unsigned short resultsL[NUM_SAMPLES];
//unsigned short resultsR[NUM_SAMPLES];
```

```c

// Use otherwise
unsigned short resultsL[1];
unsigned short resultsR[1];

void Run_ADC() {
  // Write NUM_BITS and MAX_CYCLES to ADC
  *ADC_NUM_BITS_PTR = 16;
  *ADC_MAX_CYCLES_PTR = MAX_CYCLES;

  rConv = *ADC_CONV_PTR;
  while (rConv == 1) {
    rConv = *ADC_CONV_PTR;
  }

  while (rConv == 0) {
    rConv = *ADC_CONV_PTR;
  }

  for (int i = 0; i < 2 * NUM_SAMPLES; i++) {
    if (i % 2 == 0)
      *ADC_SDI_DATA_PTR = 0x8000;
    else
      *ADC_SDI_DATA_PTR = 0xC000;

    while (rConv == 1) {
      rConv = *ADC_CONV_PTR;
    }
    while (rConv == 0) {
      rConv = *ADC_CONV_PTR;
    }

    if (i % 2 == 1) {
      resultsL[i / 2] = *ADC_SDO_DATA_PTR;
    } else {
      resultsR[i / 2] = *ADC_SDO_DATA_PTR;
    }
  }

  // Print resultsL

  xil_printf("Printing resultsL \n");
  for (int i = 0; i < NUM_SAMPLES; i++) {
    xil_printf("%d \n", resultsL[i]);
  }

  xil_printf("Printing resultsR \n");
  for (int i = 0; i < NUM_SAMPLES; i++) {
    xil_printf("%d \n", resultsR[i]);
  }

}

void Run_DAC() {
  int curr_sample;

  xil_printf("DAC START! \n");

  //Initialize SPI.
  *DAC_NUM_BITS_PTR = 24;
  *DAC_MAX_CYCLES_PTR = MAX_CYCLES;

  // Continuously output the sample sinwaves.
  while (1) {
```

```
    for (int i = 0; i < 2 * NUM_SAMPLES; ++i) {

      if (i % 2 == 0) {
        // Add half of Vcc
        curr_sample = sine_1[i] + 32768;

        // Clear upper bits
        curr_sample = curr_sample & 0x0000FFFF;

        // Add control bits
        curr_sample = curr_sample | 0x00300000;

      } else {
        // Add half of Vcc
        curr_sample = sine_2[i] + 32768;

        // Clear upper bits
        curr_sample = curr_sample & 0x0000FFFF;

        // Add control bits
        curr_sample = curr_sample | 0x00310000;
      }

      //Write the sample to the SPI
      *DAC_SDI_DATA_PTR = curr_sample;

      //Wait for the SPI to be inactive before writing the next sample.
      rConv = *DAC_CONV_PTR;
      while (rConv == 1) {
        rConv = *DAC_CONV_PTR;
      }

      while (rConv == 0) {
        rConv = *DAC_CONV_PTR;
      }
    }
  }

  xil_printf("DONE! \n");
}

void Run_ADC_to_DAC() {
  int ADC_CONV;
  int curr_sample;
  int L_sample;
  int R_sample;

  // Initialize the SPIs
  *ADC_NUM_BITS_PTR = 16;
  *DAC_NUM_BITS_PTR = 24;

  *ADC_MAX_CYCLES_PTR = MAX_CYCLES;
  *DAC_MAX_CYCLES_PTR = MAX_CYCLES;

  // Discard the first sample
  ADC_CONV = *ADC_CONV_PTR;
  while (ADC_CONV == 1) {
    ADC_CONV = *ADC_CONV_PTR;
  }

  while (ADC_CONV == 0) {
    ADC_CONV = *ADC_CONV_PTR;
  }

```

```c
  while (1) {
    // Alternate between channels.
    for (int i = 0; i < 2; i++) {
      // Write SDI for the next sample.
      if (i == 0)
        *ADC_SDI_DATA_PTR = 0x8000;

      else
        *ADC_SDI_DATA_PTR = 0xC000;

      // Wait for ADC SPI to be inactive before writing the next sample.
      while (ADC_CONV == 1) {
        ADC_CONV = *ADC_CONV_PTR;
      }
      while (ADC_CONV == 0) {
        ADC_CONV = *ADC_CONV_PTR;
      }

      // Read ADC output
      curr_sample = *ADC_SDO_DATA_PTR;

      // Clear upper bits
      curr_sample = curr_sample & 0x0000FFFF;

      // Add control bits.
      if (i == 1) {
        L_sample = curr_sample | 0x00300000;
        *DAC_SDI_DATA_PTR = L_sample;
      } else {
        R_sample = curr_sample | 0x00310000;
        *DAC_SDI_DATA_PTR = R_sample;
      }
    }
  }

}

int main() {
  init_platform();
  //Run_ADC();
  //Run_DAC();
  Run_ADC_to_DAC();

  cleanup_platform();
  return 0;
}
```

Listing 4: SPI C driver program

```c
#define ARRAY_SIZE 1000
short sine_1[ARRAY_SIZE] = { 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
```

```
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107, 0, 4107, 8149, 12063, 15786, 19261, 22431, 25248,
    27667, 29649, 31164, 32188, 32703, 32703, 32188, 31164, 29649, 27667,
    25248, 22431, 19261, 15786, 12063, 8149, 4107, 0, -4107, -8149, -12063,
    -15786, -19261, -22431, -25248, -27667, -29649, -31164, -32188, -32703,
    -32703, -32188, -31164, -29649, -27667, -25248, -22431, -19261, -15786,
    -12063, -8149, -4107 };
```

Listing 5: Input sine wave 1.

```c
#define ARRAY_SIZE 1000
short sine_2[ARRAY_SIZE] = { 0, 8149, 15786, 22431, 27667, 31164, 32703, 32188,
    29649, 25248, 19261, 12063, 4107, -4107, -12063, -19261, -25248, -29649,
    -32188, -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786,
    22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107,
    -4107, -12063, -19261, -25248, -29649, -32188, -32703, -31164, -27667,
    -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703,
    32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063, -19261, -25248,
    -29649, -32188, -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149,
    15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063,
    4107, -4107, -12063, -19261, -25248, -29649, -32188, -32703, -31164,
    -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164,
    32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063, -19261,
    -25248, -29649, -32188, -32703, -31164, -27667, -22431, -15786, -8149,
    0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261,
    12063, 4107, -4107, -12063, -19261, -25248, -29649, -32188, -32703,
    -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667,
    31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063,
    -19261, -25248, -29649, -32188, -32703, -31164, -27667, -22431, -15786,
    -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248,
    19261, 12063, 4107, -4107, -12063, -19261, -25248, -29649, -32188,
    -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431,
    27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107,
    -12063, -19261, -25248, -29649, -32188, -32703, -31164, -27667, -22431,
    -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649,
    25248, 19261, 12063, 4107, -4107, -12063, -19261, -25248, -29649,
    -32188, -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786,
    22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107,
    -4107, -12063, -19261, -25248, -29649, -32188, -32703, -31164, -27667,
    -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703,
    32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063, -19261, -25248,
    -29649, -32188, -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149,
    15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063,
    4107, -4107, -12063, -19261, -25248, -29649, -32188, -32703, -31164,
    -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164,
    32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063, -19261,
    -25248, -29649, -32188, -32703, -31164, -27667, -22431, -15786, -8149,
    0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261,
    12063, 4107, -4107, -12063, -19261, -25248, -29649, -32188, -32703,
    -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667,
    31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063,
```

```
    -19261, -25248, -29649, -32188, -32703, -31164, -27667, -22431, -15786,
    -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248,
    19261, 12063, 4107, -4107, -12063, -19261, -25248, -29649, -32188,
    -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431,
    27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107,
    -12063, -19261, -25248, -29649, -32188, -32703, -31164, -27667, -22431,
    -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649,
    25248, 19261, 12063, 4107, -4107, -12063, -19261, -25248, -29649,
    -32188, -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786,
    22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107,
    -4107, -12063, -19261, -25248, -29649, -32188, -32703, -31164, -27667,
    -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703,
    32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063, -19261, -25248,
    -29649, -32188, -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149,
    15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063,
    4107, -4107, -12063, -19261, -25248, -29649, -32188, -32703, -31164,
    -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164,
    32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063, -19261,
    -25248, -29649, -32188, -32703, -31164, -27667, -22431, -15786, -8149,
    0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261,
    12063, 4107, -4107, -12063, -19261, -25248, -29649, -32188, -32703,
    -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667,
    31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063,
    -19261, -25248, -29649, -32188, -32703, -31164, -27667, -22431, -15786,
    -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248,
    19261, 12063, 4107, -4107, -12063, -19261, -25248, -29649, -32188,
    -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786, 22431,
    27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107, -4107,
    -12063, -19261, -25248, -29649, -32188, -32703, -31164, -27667, -22431,
    -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703, 32188, 29649,
    25248, 19261, 12063, 4107, -4107, -12063, -19261, -25248, -29649,
    -32188, -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149, 15786,
    22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063, 4107,
    -4107, -12063, -19261, -25248, -29649, -32188, -32703, -31164, -27667,
    -22431, -15786, -8149, 0, 8149, 15786, 22431, 27667, 31164, 32703,
    32188, 29649, 25248, 19261, 12063, 4107, -4107, -12063, -19261, -25248,
    -29649, -32188, -32703, -31164, -27667, -22431, -15786, -8149, 0, 8149,
    15786, 22431, 27667, 31164, 32703, 32188, 29649, 25248, 19261, 12063,
    4107, -4107, -12063, -19261, -25248, -29649, -32188, -32703, -31164,
    -27667, -22431, -15786, -8149 };
```

Listing 6: Input sine wave 2.

References

[1] Linear Technology Corporation, "LTC1864L/LTC1865L: µPower, 3V, 16-Bit, 150ksps 1- and 2-Channel ADCs in MSOP," Linear Technology Corporation, Milpitas, CA, Tech. Rep. 18645Lfs, 2001, datasheet.

[2] ——, "LTC1654: Dual 14-Bit Rail-to-Rail DAC in 16-Lead SSOP Package," Linear Technology Corporation, Milpitas, CA, Tech. Rep. 1654fb Rev. B, 2000, datasheet.